Introduction to Logic I

Joona Piirainen

Autumn 2026

Abstract

Notes for the University of Helsinki course MAT21014, taught in Finnish from 03.09.2026–17.10.2026.

Contents

1 Course information **1**

2 Lecture notes **1**

2.1 03.09.2026 1

2.1.1 Propositional formulas 1

2.1.2 Subformulas 2

2.1.3 Structural induction 2

2.2 07.09.2026 3

2.2.1 Structural induction 3

2.2.2 Closures 3

2.2.3 Semantics of propositional logic 5

2.3 10.09.2026 7

2.3.1 Logical implication and equivalence 7

1 Course information

The official course information is available on the University of Helsinki course page.

2 Lecture notes

2.1 03.09.2026

2.1.1 Propositional formulas

Definition 2.1. The set of *propositional formulas* is defined inductively as follows:

1. Every propositional symbol p_n, where $n \in \mathbb{N}$, is a propositional formula.

2. If A and B are propositional formulas, then

$$\neg A, \qquad (A \wedge B), \qquad (A \vee B), \qquad (A \to B), \qquad (A \leftrightarrow B)$$

are propositional formulas.

3. No other strings are propositional formulas.

2.1.2 Subformulas

Definition 2.2 (Immediate subformulas)**.** The immediate subformulas of a propositional formula are defined by its outermost connective:

1. A propositional symbol p_n has no immediate subformulas.
2. The only immediate subformula of $\neg A$ is A.
3. The immediate subformulas of $(A \circ B)$ are A and B, where

$$\circ \in \{\wedge, \vee, \to, \leftrightarrow\}.$$

Definition 2.3 (Subformulas)**.** A formula B is a *subformula* of a formula A if $B = A$, or if there are formulas

$$A = A_0, A_1, \ldots, A_k = B$$

such that $k \geq 1$ and A_{i+1} is an immediate subformula of A_i for every $0 \leq i < k$.

2.1.3 Structural induction

Theorem 2.4 (Structural induction)**.** *Let $P(A)$ be a statement about propositional formulas. Suppose that*

1. *$P(p_n)$ holds for every $n \in \mathbb{N}$;*
2. *if $P(A)$ holds, then $P(\neg A)$ holds; and*
3. *if $P(A)$ and $P(B)$ hold, then $P((A \circ B))$ holds for every*

$$\circ \in \{\wedge, \vee, \to, \leftrightarrow\}.$$

Then $P(A)$ holds for every propositional formula A.

Example 2.5 (Structural induction)**.** For a formula A, let $v(A)$ be the number of occurrences of propositional symbols in A, and let $b(A)$ be the number of occurrences of binary connectives in A. We prove that

$$v(A) = b(A) + 1$$

for every propositional formula A.

If $A = p_n$, then $v(A) = 1$ and $b(A) = 0$, so the claim holds. Suppose next that the claim holds for a formula A. Negation introduces neither a propositional symbol nor a binary connective, and hence

$$v(\neg A) = v(A) = b(A) + 1 = b(\neg A) + 1.$$

Finally, suppose that the claim holds for formulas A and B, and let $\circ$ be any binary connective. Then

$$\begin{aligned} v((A \circ B)) &= v(A) + v(B) \\ &= (b(A) + 1) + (b(B) + 1) \\ &= b((A \circ B)) + 1. \end{aligned}$$

The result follows by structural induction. □

2.2 07.09.2026

2.2.1 Structural induction

Example 2.6 (Matching parentheses)**.** For a propositional formula A, let $l(A)$ and $r(A)$ denote the numbers of occurrences of left and right parentheses in A, respectively. We prove by structural induction that

$$l(A) = r(A)$$

for every propositional formula A.

If $A = p_n$, then $l(A) = 0 = r(A)$. Suppose next that the claim holds for a formula A. Negation introduces no parentheses, so

$$l(\neg A) = l(A) = r(A) = r(\neg A).$$

Finally, suppose that the claim holds for formulas A and B, and let $\circ$ be any binary connective. The formula $(A \circ B)$ introduces one left and one right parenthesis. Hence

$$\begin{aligned} l((A \circ B)) &= l(A) + l(B) + 1 \\ &= r(A) + r(B) + 1 \\ &= r((A \circ B)). \end{aligned}$$

The result follows by structural induction. □

2.2.2 Closures

Definition 2.7 (Closure)**.** Let X be a set, let $B \subseteq X$, and, for $1 \leq i \leq k$, let

$$f_i \colon X^{n_i} \longrightarrow X$$

be a function. A subset $Y \subseteq X$ is *closed under* f_i if

$$f_i(y_1, \ldots, y_{n_i}) \in Y$$

whenever $y_1, \ldots, y_{n_i} \in Y$. The *closure of* B *under* $f_1, \ldots, f_k$, denoted by

$$\mathrm{cl}(B; f_1, \ldots, f_k),$$

is the smallest subset of X that contains B and is closed under every f_i.

Equivalently, the closure can be written as the intersection

$$\mathrm{cl}(B; f_1, \ldots, f_k) = \bigcap \{Y \subseteq X : B \subseteq Y \text{ and } Y \text{ is closed under every } f_i\}.$$

Example 2.8. Let $X = \mathbb{Z}$, $B = \{1\}$, and define functions $f, g \colon \mathbb{Z} \to \mathbb{Z}$ by

$$f(x) = x + 1 \qquad \text{and} \qquad g(x) = -x.$$

Then

$$\mathrm{cl}(\{1\}; f, g) = \mathbb{Z}.$$

Indeed, every subset of $\mathbb{Z}$ that contains 1 and is closed under g must also contain -1. Closure under f then gives 0, and repeated applications of f give every positive integer. Applying g to these integers gives every negative integer. Thus any such closed set is all of $\mathbb{Z}$, while $\mathbb{Z}$ itself clearly contains 1 and is closed under both functions.

Corollary 2.9 (Induction on a closure)**.** *Set*

$$C = \mathrm{cl}(B; f_1, \ldots, f_k).$$

To prove that every element of C satisfies a property P, it suffices to prove that

1. *every element of B satisfies P; and*
2. *for each i, if $x_1, \ldots, x_{n_i}$ satisfy P, then $f_i(x_1, \ldots, x_{n_i})$ also satisfies P.*

Proof. Let Y be the set of all elements of $\mathrm{cl}(B; f_1, \ldots, f_k)$ that satisfy P. The two assumptions imply that $B \subseteq Y$ and that Y is closed under every f_i. Since the closure is the smallest such set,

$$\mathrm{cl}(B; f_1, \ldots, f_k) \subseteq Y.$$

Hence every element of the closure satisfies P. □

Let Σ be the alphabet of propositional logic, let Σ^* be the set of all finite strings over Σ, and let

$$V = \{p_n : n \in \mathbb{N}\}$$

be the set of propositional symbols. The logical connectives determine the following functions on strings:

$$f_\neg(A) = \neg A$$

and

$$f_\circ(A, B) = (A \circ B), \qquad \circ \in \{\wedge, \vee, \to, \leftrightarrow\}.$$

The set of propositional formulas is precisely

$$\mathrm{cl}(V; f_\neg, f_\wedge, f_\vee, f_\to, f_\leftrightarrow).$$

In particular, if $S \subseteq \Sigma^*$ contains every propositional symbol and is closed under all functions associated with the logical connectives, then S contains every propositional formula.

2.2.3 Semantics of propositional logic

Definition 2.10 (Truth assignment)**.** A *truth assignment* is a function

$$v\colon V \longrightarrow \{0,1\},$$

where V is the set of propositional symbols. The values 0 and 1 represent **false** and **true**, respectively.

The truth values of the logical connectives are defined as follows.

Definition 2.11 (Negation)**.** The negation $\neg A$ is true exactly when A is false.

A	$\neg A$
0	1
1	0

Definition 2.12 (Conjunction)**.** The conjunction $(A \wedge B)$ is true exactly when both A and B are true.

A	B	$(A \wedge B)$
0	0	0
0	1	0
1	0	0
1	1	1

Definition 2.13 (Disjunction)**.** The disjunction $(A \vee B)$ is true exactly when at least one of A and B is true.

A	B	$(A \vee B)$
0	0	0
0	1	1
1	0	1
1	1	1

Definition 2.14 (Implication)**.** The implication $(A \to B)$ is false exactly when A is true and B is false. It is important to note that implication is **not** symmetric.

A	B	$(A \to B)$
0	0	1
0	1	1
1	0	0
1	1	1

Definition 2.15 (Equivalence)**.** The equivalence $(A \leftrightarrow B)$ is true exactly when A and B have the same truth value.

A	B	$(A \leftrightarrow B)$
0	0	1
0	1	0
1	0	0
1	1	1

Definition 2.16 (Truth value)**.** Let A be a propositional formula and let v be a truth assignment. The *truth value* $v(A)$ of A is defined by the following rules:

1. If A is a propositional symbol p_n, then $v(A)$ is already defined.
2. If $A = \neg B$ and $v(B)$ is defined, then
$$v(A) = \begin{cases} 0, & \text{if } v(B) = 1, \\ 1, & \text{if } v(B) = 0. \end{cases}$$
Another way to write this is $v(A) = 1 - v(B)$.
3. If $A = (B \wedge C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ exactly when $v(B) = v(C) = 1$, and $v(A) = 0$ otherwise. Another way to write this is $v(A)v(B)$.
4. If $A = (B \vee C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ when at least one of $v(B) = 1$ and $v(C) = 1$ holds, and $v(A) = 0$ when $v(B) = v(C) = 0$.
5. If $A = (B \to C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 0$ when $v(B) = 1$ and $v(C) = 0$, and $v(A) = 1$ in all other cases.
6. If $A = (B \leftrightarrow C)$ and $v(B)$ and $v(C)$ are defined, then $v(A) = 1$ iff $v(B) = v(C)$.

Definition 2.17 (Classification of propositional formulas)**.** A propositional formula A is

1. a *tautology* if $v(A) = 1$ for every truth assignment v;
2. a *contradiction* if $v(A) = 0$ for every truth assignment v;
3. *contingent* if there are truth assignments v and w such that $v(A) = 1$ and $w(A) = 0$;
4. *satisfiable* if $v(A) = 1$ for at least one truth assignment v; and
5. *falsifiable* if $v(A) = 0$ for at least one truth assignment v.

Example 2.18 (De Morgan's laws)**.** Consider the formulas

$$(\neg(A \wedge B) \leftrightarrow (\neg A \vee \neg B)) \qquad \text{and} \qquad (\neg(A \vee B) \leftrightarrow (\neg A \wedge \neg B)).$$

Their truth values can be compared using the following table:

A	B	$\neg(A \wedge B)$	$(\neg A \vee \neg B)$	$\neg(A \vee B)$	$(\neg A \wedge \neg B)$
0	0	1	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	0	0	0

The two sides of each equivalence have the same truth value under every truth assignment. Consequently, both equivalences are tautologies. These equivalences are known as *De Morgan's laws*.

2.3 10.09.2026

2.3.1 Logical implication and equivalence

Definition 2.19 (Logical implication)**.** A formula A *logically implies* a formula B, written

$$A \Rightarrow B,$$

if, for every truth assignment v, $v(A) = 1$ implies $v(B) = 1$. Equivalently, $A \Rightarrow B$ exactly when $(A \to B)$ is a tautology.

Definition 2.20 (Logical equivalence)**.** Formulas A and B are *logically equivalent*, written

$$A \Leftrightarrow B,$$

if $v(A) = v(B)$ for every truth assignment v. Equivalently, $A \Leftrightarrow B$ exactly when $(A \leftrightarrow B)$ is a tautology.

The symbols $\Rightarrow$ and $\Leftrightarrow$ are used in the metalanguage to state relations between formulas; unlike $\to$ and $\leftrightarrow$, they are not connectives used to construct propositional formulas.

Definition 2.21 (Literal)**.** A *literal* is either a propositional symbol p_n or its negation $\neg p_n$. The former is a *positive literal*, and the latter is a *negative literal*.

Definition 2.22 (Disjunctive normal form)**.** A formula is in *disjunctive normal form* (DNF) if it is a disjunction of conjunctions of literals. In other words, it has the form

$$\bigvee_{i=1}^{m} \left(\bigwedge_{j=1}^{k_i} L_{ij} \right),$$

where $m \geq 1$, each $k_i \geq 1$, and every L_{ij} is a literal.

Definition 2.23 (Conjunctive normal form)**.** A formula is in *conjunctive normal form* (CNF) if it is a conjunction of disjunctions of literals. In other words, it has the form

$$\bigwedge_{i=1}^{m} \left(\bigvee_{j=1}^{k_i} L_{ij} \right),$$

where $m \geq 1$, each $k_i \geq 1$, and every L_{ij} is a literal.

For a literal L, let $\overline{L}$ denote its *complementary literal*, defined by

$$\overline{p_n} = \neg p_n \qquad \text{and} \qquad \overline{\neg p_n} = p_n.$$

De Morgan's laws show that DNF and CNF are dual: negating a DNF formula gives an equivalent CNF formula, and negating a CNF formula gives an equivalent DNF formula. More precisely,

$$\neg \bigvee_{i=1}^{m} \left(\bigwedge_{j=1}^{k_i} L_{ij} \right) \Leftrightarrow \bigwedge_{i=1}^{m} \left(\bigvee_{j=1}^{k_i} \overline{L_{ij}} \right),$$

and the analogous equivalence holds with $\wedge$ and $\vee$ interchanged.

Definition 2.24 (Boolean function)**.** An n-ary *truth function* is a function

$$f\colon \{0,1\}^n \longrightarrow \{0,1\}.$$

A propositional formula A whose propositional symbols are among $p_1, \ldots, p_n$ determines the truth function

$$f_A(x_1, \ldots, x_n) = v(A),$$

where $v(p_i) = x_i$ for every $1 \leq i \leq n$.

Theorem 2.25 (Normal form theorem)**.** *Let f be an n-ary truth function, where $n \geq 1$. Then there are formulas A_D in DNF and A_C in CNF, containing only the propositional symbols $p_1, \ldots, p_n$, such that*

$$f_{A_D} = f_{A_C} = f.$$

Consequently, every propositional formula is logically equivalent to a formula in DNF and to a formula in CNF.

Proof. We construct the formulas directly from the truth table of f.

To construct A_D, consider each row $a = (a_1, \ldots, a_n) \in \{0,1\}^n$ on which $f(a) = 1$, and define the formula

$$M_a = L_1^a \wedge \cdots \wedge L_n^a, \qquad L_i^a = \begin{cases} p_i, & \text{if } a_i = 1, \\ \neg p_i, & \text{if } a_i = 0. \end{cases}$$

Thus M_a is true under a truth assignment v exactly when $(v(p_1), \ldots, v(p_n)) = a$.

If there is at least one such row, let

$$A_D = \bigvee_{\substack{a \in \{0,1\}^n \\ f(a)=1}} M_a.$$

Under an assignment corresponding to $b \in \{0,1\}^n$, the formula A_D is true exactly when the disjunction contains M_b, which happens exactly when $f(b) = 1$. If f is constantly false, take $A_D = (p_1 \wedge \neg p_1)$. Thus $f_{A_D} = f$.

Dually, to construct A_C, consider each row a on which $f(a) = 0$ and define

$$C_a = K_1^a \vee \cdots \vee K_n^a, \qquad K_i^a = \begin{cases} p_i, & \text{if } a_i = 0, \\ \neg p_i, & \text{if } a_i = 1. \end{cases}$$

The clause C_a is false exactly on the row a. If there is at least one such row, let

$$A_C = \bigwedge_{\substack{a \in \{0,1\}^n \\ f(a)=0}} C_a.$$

This conjunction is true on a row b exactly when $f(b) = 1$. If f is constantly true, take $A_C = (p_1 \vee \neg p_1)$. Thus $f_{A_C} = f$.

All finite conjunctions and disjunctions above may be given any fixed binary parenthesization. Finally, for any propositional formula, list its distinct propositional symbols as $q_1, \ldots, q_n$ and apply the same construction with q_i in place of p_i. This gives logically equivalent formulas in DNF and CNF. □

Example 2.26 (Constructing normal forms from a truth function)**.** Consider the binary truth function whose truth table is

x_1	x_2	$f(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

The rows where $f = 1$ are $(0, 1)$ and $(1, 0)$. Replace a 1 in a row by p_i, replace a 0 by $\neg p_i$, and join the literals in each row with $\wedge$. This gives the formulas

$$M_{(0,1)} = (\neg p_1 \wedge p_2) \qquad \text{and} \qquad M_{(1,0)} = (p_1 \wedge \neg p_2).$$

Finally, join the formulas for the selected rows with $\vee$:

$$A = ((\neg p_1 \wedge p_2) \vee (p_1 \wedge \neg p_2)).$$

This is a DNF formula with $f_A = f$.

To construct a CNF formula, use instead the rows $(0, 0)$ and $(1, 1)$, where $f = 0$. The clauses that are false on exactly these rows are

$$C_{(0,0)} = (p_1 \vee p_2) \qquad \text{and} \qquad C_{(1,1)} = (\neg p_1 \vee \neg p_2).$$

Their conjunction

$$B = ((p_1 \vee p_2) \wedge (\neg p_1 \vee \neg p_2))$$

is in CNF and satisfies $f_B = f$. Thus $A \Leftrightarrow B$.